

00_codebase_inventory

HelixCode LLM-Serving Stack — Grounded Codebase Inventory

Revision: 1 **Last modified:** 2026-07-06 **Maintainer:** codebase-reconnaissance subagent **Scope:** Factual map of the current LLM-serving stack for detailed implementation planning. **Method:** Every claim below is grounded in a real file path + line read this session. Anything not found is marked **ABSENT** with the paths checked (§11.4.6 no-guessing).

Repo root: /home/milos/Factory/projects/tools_and_research/
helix_code

1. HelixLLM

LOCATED. HelixLLM is a **full standalone Go submodule**, present at TWO paths (both real, both populated with identical content — the flat + grouped layouts per CONST-051(C)):

- submodules/helix_llm/ (primary working path this session)
- dependencies/HelixDevelopment/helix_llm/ (second checkout of the same repo)

Facts: - **Module id / language:** module github.com/HelixDevelopment/
HelixLLM, go 1.26.1 — submodules/helix_llm/go.mod:1-3. - **Upstream:**
git@github.com:HelixDevelopment/HelixLLM.git (not in root .gitmodules
under that name — the root .gitmodules wires dependencies/
HelixDevelopment/* for doc_processor, llm_orchestrator, llm_provider,
vision_engine, llms_verifier, but NOT helix_llm; the submodules/
helix_llm + dependencies/HelixDevelopment/helix_llm checkouts exist on
disk — see GAPS). - **What it is** (submodules/helix_llm/README.md:1-20):
“Enterprise-grade distributed LLM system built in Go with Gin Gonic. A single
binary with a mode system.” It provides: - **OpenAI-compatible API** (/v1/chat/
completions, /v1/completions, /v1/models, /v1/embeddings) AND
Anthropic-compatible API (/v1/messages) — README API table. - **Local
inference via llama.cpp** (CUDA/Metal/ROCM). - **Multi-provider fallback chain** —
auto-discovers free models from 7+ cloud providers (Chutes, OpenRouter,
HuggingFace, Nvidia, Cerebras, SambaNova, Together), scores via LLMsVerifier,
routes ranked chain with 429/5xx failover, **llama.cpp as guaranteed last resort.** -

RAG pipeline, ReAct agent system w/ tool calling, HTTP/3 server, mode system (full, gateway, brain, knowledge, agents, control). - **Entry point:** submodules/helix_llm/cmd/helixllm/main.go (single binary). Internal packages: internal/

{agents,brain,control,fallback,gateway,knowledge,mode,server,shared,testing}(ls internal/). - **The LLM coordination layer is internal/brain/** — defines Provider interface + Router + Brain service (internal/brain/provider.go:1-6 package doc).

1.1 HelixLLM Provider interface (internal/brain/provider.go:14-35)

```
type Provider interface {
    Complete(ctx context.Context, req
        *types.InternalChatRequest)
        (*types.InternalChatResponse, error)
    CompleteStream(ctx context.Context, req
        *types.InternalChatRequest) (<-chan types.StreamChunk,
        error)
    Models() []string
    Name() string
    Available() bool
}
```

1.2 HelixLLM provider implementations (internal/brain/*_provider.go + llama.cpp.go)

Present, one file each: anthropic_provider.go, openai_provider.go, openai_compat_provider.go, cerebras_provider.go, chutes_provider.go, huggingface_provider.go, nvidia_provider.go, openrouter_provider.go, sambanova_provider.go, together_provider.go, and llama.cpp.go.

1.3 llama.cpp provider (internal/brain/llamacpp.go)

```
type LlamaCppProvider struct { baseURL string; models []string;
    client *http.Client; registry *models.Registry }
func NewLlamaCppProvider(baseURL string, models []string)
    *LlamaCppProvider // :29
func (p *LlamaCppProvider) Name() string { return
    "llamacpp" } // :39
func (p *LlamaCppProvider) Available()
    bool // :49 GET
    {base}/health
```

```

func (p *LlamaCppProvider)
    Complete(...) //
    :67 POST {base}/v1/chat/completions
func (p *LlamaCppProvider)
    CompleteStream(...) //
    :103 SSE

```

It talks to llama.cpp's **OpenAI-compatible server** at a configurable base URL (`http://host:port`). It is a **real HTTP client** — no simulation.

1.4 Wiring in the binary (`cmd/helixllm/main.go`)

- `brain.New(brain.Config{ LlamaCppURL: "http://<LocalRPCHost>:<LocalRPCPort>", LlamaCppModels: []string{cfg.LLM.LocalModel}, OpenAIKey, AnthropicKey, ChutesKey, OpenRouterKey, HuggingFaceKey, NvidiaKey, CerebrasKey, SambaNovaKey, TogetherKey, ... Registry, KVCache })` — `main.go:268-285`. Brain **registers whichever providers are configured** (`brain.go:95-126`, one `if cfg.XxxKey != ""` block per cloud provider; `RegisterProvider` at `brain.go:159`).
- `brain.NewLlamaServer(brain.LlamaServerConfig{...})` — `main.go:220` (spawns/manages the local llama.cpp server process).
- Fallback chain: `fallback.NewScorerBridge{VerifierURL: cfg.LLM.VerifierURL, RefreshInterval} → fallback.NewChain(brainSvc.Providers(), rateLimiter) → scorerBridge.StartRefreshLoop(...)` — `main.go:328-345`. **"llamacpp always placed last as the local safety net"** (`main.go:327` comment).
- Embeddings: `knowledge.NewEmbedder(cfg.Knowledge.EmbeddingProvider, ..., 768)` with providers "llama" / "openai" and a hash-embedder fallback — `main.go:290-320`. Vector store: `knowledge.NewVectorStore(cfg.Knowledge.VectorDB, "localhost", 6333)` (Qdrant, `internal/knowledge/qdrant.go`).

GAPS (\$1): - No **vision** or **translation** provider/endpoint in HelixLLM (`grep -rlniE 'vision|translat' internal/` returns only `config/i18n/analytics/registry` files, no vision serving surface). - No `.gitmodules` entry for `helix_llm` at the meta-repo root → its version is not pinned/tracked the way other own-org submodules are (checked root `.gitmodules`). - llama.cpp base URL + model list are **config-driven single values** (`LocalModel`, `LocalRPCHost/Port`) — no multi-model / multi-GGUF registry populated from disk shown in `main.go` (registry exists at `internal/brain/models/registry.go` but wiring to a GGUF directory is via `brain.NewDownloader(cfg.LLM.ModelsDir)` at `main.go:182`; no RTX-5090 / GPU-layer configuration surface found). - MCP/ACP capability serving: HelixLLM has `agents/mcp*.go` (tool bridge) but no ACP server found.

2. HelixAgent (submodules/helix_agent/)

- **Module id:** module dev.helix.agent, go 1.26 (go.mod:1-3). Upstream git@github.com:HelixDevelopment/HelixAgent.git (root .gitmodules, path = submodules/helix_agent).
- ~70 internal/ packages (ls internal/), including llm/, providers/ (via llm/providers/), router/, services/, handlers/, adapters/, verifier/, rag/, mcp/, vectordb/, embeddings/.

2.1 Provider abstraction (internal/llm/provider.go:9-15)

```
type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest)
        (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest)
        (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
    ValidateConfig(config map[string]interface{}) (bool,
        []string)
}
```

NOTE: this is a **different, richer interface** than HelixLLM's brain.Provider (\$1.1). HelixAgent is the consumer-facing agent platform; HelixLLM is a backend it can call.

2.2 Implemented providers (internal/llm/providers/<name>/) — ~45 directories

ai21, anthropic, anthropic_cu, azure, cerebras, chutes, claude, cloudflare, codestral, cohere, common, deepseek, fireworks, gemini, generic, githubmodels, groq, helixllm, huggingface, hyperbolic, junie, kilo, kimi, kimicode, lmstudio, mistral, modal, nia, nlpcloud, novita, nvidia, ollama, openai, openrouter, perplexity, publicai, qwen, replicate, sambanova, sarvam, siliconflow, together, upstage, venice, vertex, vulavula (ls internal/llm/providers/).

- **helixllm provider** = the bridge to the HelixLLM backend (\$1): internal/llm/providers/helixllm/provider.go.
 - defaultEndpoint = "https://localhost:8443", endpoints /v1/chat/completions, /v1/embeddings, /v1/models, /internal/health (provider.go:23-31).

- `Config{ Endpoint, APIKey, Model, Timeout, TLSSkipVerify, UseLlamaCpp };` `UseLlamaCpp` toggles HelixLLM's local llama.cpp backend, sourced from `HELIX_LLM_USE_LLAMACPP` env, communicated via `X-Helix-LLM-Use-LlamaCpp` header (`provider.go:49-88`).
- A second **adapter** exists at `internal/adapters/helixllm/adapter.go` (+ `types.go`).

2.3 Provider registry (`internal/services/provider_registry.go`)

- `NewProviderRegistry(cfg, memory) → registerDefaultProviders(cfg) (:324, :697)` + **auto-discovery** from env (`NewProviderDiscovery, :395-441`).
- `createProviderFromConfig switch (:1535+):` case "claude", "deepseek", "gemini", "helixllm" (:1595 — instantiates `helixllm.NewProvider(config)`), and more.
- `InitializeFromStartupVerifier` registers providers **from LLMsVerifier verified results** (:588).
- The verification service is wired to the registered providers for real API calls (:473-478).

2.4 OpenAI-/Anthropic-compatible REST server (`internal/handlers/openai_compatible.go`)

- Routes: `protected.POST("/chat/completions", h.ChatCompletions) + .../stream (:379-380)`.
- Exposes `GET /v1/models` with a canonical model list (:531, :2300) including synthetic aliases like `helixagent-debate / helixagent-llm` (:574).
- Google-compatible + Anthropic-compatible handlers also present (`internal/handlers/google_compatible.go, completion* handlers`). → **HelixAgent DOES expose an OpenAI-compatible server that CLI agents can point at.**

2.5 HelixLLM integration docs (real)

`submodules/helix_agent/docs/HELIXLLM_INTEGRATION.md`, `HELIXLLM_USER_MANUAL.md`, `HELIXLLM_TESTING_GUIDE.md`; `docker-compose.helixllm.yml` + `docker-compose.helixllm-infra.yml`; dozens of per-CLI-agent E2E challenge scripts `challenges/scripts/cli_agent*_helixllm_e2e_challenge.sh` (aider, plandex, gptme, mistral-code, amazon-q, opencode, etc.).

GAPS (\$2): - No vision/translation/embedding-dedicated provider directory under `internal/llm/providers/` (embeddings handled via `internal/embeddings/` + HelixLLM `/v1/embeddings`; no standalone translation surface). - Two parallel HelixLLM integration seams (`internal/llm/providers/helixllm/` AND `internal/adapters/helixllm/`) — potential duplication to reconcile.

3. LLMsVerifier (submodules/llms_verifier/)

- **Module id:** `module llmsverifier, go 1.25.3 (go.mod)`. Upstream `git@github.com:vasic-digital/LLMsVerifier.git` (root `.gitmodules`, `path = submodules/llms_verifier`).
- Inner Go module tree lives under `llm-verifier/` (e.g. `llm-verifier/llmverifier/`, `llm-verifier/api/`, `llm-verifier/providers/`, `llm-verifier/sdk/go/`).
- Also has ACP docs (`ACP_*.md`), a Go SDK (`sdk/go/client.go`), `k8s/helm`, and a challenges bank.

3.1 VerificationResult (llm-verifier/llmverifier/models.go:10+)

```
type VerificationResult struct {
    ...
    CodeCapabilities          CodeCapabilityResult
    GenerativeCapabilities    GenerativeCapabilityResult
    PerformanceScores        PerformanceScore
    ScoreDetails              ScoreDetails
    Capabilities              Capabilities    // (on the model/
    provider record, models.go:33)
    ...
}
```

3.2 Capabilities (models.go:83-93)

```
type Capabilities struct {
    Completion, Chat, Embedding, FineTuning, ImageGeneration,
    CodeGeneration,
    ToolUse, Multimodal, FunctionCalling, Voice, Rerank bool
}
```

Also a `FeatureResult`-style struct with Embeddings, MCPs, etc. (`models.go:184-189`). `ContextWindow` has a custom `UnmarshalJSON` tolerant of both object and bare-int forms (`Groq`) (`models.go:96-120`). `PerformanceScore` = `OverallScore/CodeCapability/...` ; `CodeCapabilityBreakdown` = `generation/`

completion/ debugging/review/testgen/document/architecture/optimization scores (models.go:266-291).

3.3 Providers it knows / verifies

Discovered via endpoint-URL matching (config_export.go:78-81):
api.anthropic.com, api.openai.com, api.deepseek.com, api.groq.com/
openai/v1, gemini. Integration tests exercise OpenAI, Anthropic, Groq, Gemini
(integration_test.go). It performs **real HTTP feature detection** (HTTP/3,
Brotli, etc. — feature_detection_test.go). Config export produces per-agent
config (crush/opencode verifier sub-packages: llm-verifier/pkg/crush/
verifier/, llm-verifier/pkg/opencode/verifier/).

3.4 Integration point with HelixLLM / HelixAgent

- HelixLLM's fallback.ScorerBridge polls cfg.LLM.VerifierURL and re-orders the fallback chain by verifier score (helix_llm/cmd/helixllm/main.go:328-345).
- HelixAgent's provider registry initializes providers from StartupVerifier verified results (provider_registry.go:588). This is the CONST-036/037 single-source-of-truth wiring.

GAPS (\$3): - Capabilities has no explicit MCP/LSP/ACP/RAG/Skills/Plugins boolean at the model level (only MCPs on a separate FeatureResult struct at models.go:189); CONST-040 wants MCP/LSP/ACP/Embedding/RAG/Skills/Plugins flags sourced from VerificationResult — partial today. - No llama.cpp/ollama local-provider verification path surfaced in config_export.go endpoint matcher (local models are not URL-matched to a known cloud provider).

4. llama.cpp / GGUF / Ollama / vLLM usage across the tree

- **llama.cpp source** vendored as a submodule: dependencies/LLama_CPP/ (upstream git@github.com:ggml-org/llama.cpp.git, branch master; pinned a4107133 (gguf-v0.19.0-827) per git submodule status). Full CMake source tree present (CMakeLists.txt, CMakePresets.json, ci/, cmake/, ...).
- **Ollama** vendored: dependencies/Ollama/ (upstream git@github.com:ollama/ollama.git, pinned 964ea42c v0.13.4-rc2-613).
- **HuggingFace Hub** vendored: dependencies/HuggingFace_Hub/ (.gitmodules).

- **How llama.cpp is invoked/wired today:**
 - HelixLLM internal/brain/llamacpp.go = HTTP client to a **running llama.cpp OpenAI-compatible server** (§1.3).
 - HelixLLM brain.NewLlamaServer(...) (cmd/helixllm/main.go:220) manages a local llama-server process; brain.NewDownloader(cfg.LLM.ModelsDir) handles GGUF model download (main.go:182).
 - There is a dedicated challenge helix_agent/challenges/scripts/helixllm_llamacpp_only_challenge.sh.
- **vLLM: ABSENT** — no reference found (grep -rln vLLM/vllm not present in read paths; not in dependencies, not in helix_llm/helix_agent provider lists).

GAPS (§4): - No RTX-5090 / CUDA GPU-layer / tensor-split / -ngl configuration surface located in HelixLLM's llama-server launch config (only host/port/model/models-dir shown). Needs verification of brain.LlamaServerConfig fields (in internal/brain/server.go, not fully read). - The vendored dependencies/LLama_CPP source is present but no build/target wiring found that compiles it for the host GPU (setup path uses a running server, not a local build recipe surfaced this session).

5. containers submodule (submodules/containers/)

- **Module id:** module digital.vasic.containers,go 1.25.0 (go.mod). Upstream git@github.com:vasic-digital/Containers.git (root .gitmodules,path = submodules/containers).
- **Public pkg/ API** (ls pkg/): boot, compose, health, orchestrator, runtime, lifecycle, discovery, endpoint, serviceregistry, network, volume, metrics, monitor, emulator, vm, remote, remoteexec, distribution, policy, egress, envconfig, event, i18n, lazyservice, logging, scheduler, applesim, genymotion, cuttlefish, crossbuild, ctop, brokertest, cache.

5.1 Boot API (pkg/boot/manager.go)

```
func NewBootManager(endpoints
    map[string]endpoint.ServiceEndpoint,
    opts ...BootManagerOption) *BootManager // :41
func (bm *BootManager) BootAll(ctx ...) ... // :64
    discovery → compose up → health checks → summary
func (bm *BootManager) HealthCheckAll(...) // :291
func (bm *BootManager) Shutdown(ctx) error // :330
```


Consumer declares services as a `map[string]endpoint.ServiceEndpoint`, passes options (logger, metrics), and calls `BootAll` — the **on-demand-infra invariant** of §11.4.76.

5.2 Compose API (`pkg/compose/`)

`group.go`, `orchestrator.go`, `options.go`, `types.go`, **`helix_project.go`** (a Helix-specific compose project helper), `podman_compose_test.go` (Podman-oriented — matches §11.4.161 rootless-Podman mandate).

5.3 Health API (`pkg/health/`)

`checker.go`, `http.go`, `tcp.go`, `grpc.go`, `gpu.go` (**GPU health check present** — relevant to RTX-5090), `custom.go`, `retry.go`, **`helix_infra.go`** (Helix infra health helper).

GAPS (§5): none blocking — the boot/compose/health triad exists and is Podman-aware. To run a llama.cpp server container via this submodule, a consumer would register a `ServiceEndpoint` + compose group; no pre-built llama.cpp service definition was found in `pkg/compose` (would be a new project-side registration or an upstream extension per §11.4.74).

6. HelixQA (`submodules/helix_qa/`)

- Upstream `git@github.com:HelixDevelopment/HelixQA.git` (root `.gitmodules`, `path = submodules/helix_qa`). Go module + `go.work`. Has `helix-deps.yaml` (CONST-054 manifest).
- **Test banks:** `banks/*. {json,yaml}` — e.g. `admin-operations`, `app-navigation`, `all-formats`, `aichat-bash-tools-comprehensive`, `atmosphere_additions_*`. A new capability registers a **test bank** as a `banks/<name>. {yaml,json}` pair.
- **Binaries / runners:** `cmd/` has 20+ tools: `helixqa` (main), `helixqa-bank-session`, `helixqa-concrete-runner`, `helixqa-bridge`, `helixqa-recvalidate` (recording validation), `helixqa-omniparser`, `helixqa-uitars`, `helixqa-lpips/helixqa-dreamsim` (perceptual diff), `helixqa-text`, `helixqa-x11grab/helixqa-kmsggrab/helixqa-capture-linux` (screen capture), `recording-analyzer`, `qa-audio-probe`, `ocu-probe`.
- Also `banks/`, `challenges/`, `pkg/`, `internal/`, `tests/`, `monitoring/`, `docker-compose.stack.yml`, `ARCHITECTURE.md`, `API_REFERENCE.md`, `OPENCV_INTEGRATION_ARCHITECTURE.md`.
- **Sync-channel / evidence format:** the recording/vision toolchain (`recording-analyzer`, `recvalidate`, `omniparser`, `uitars`, `lpips/dreamsim`) is the §11.4.107/§11.4.117/§11.4.160 vision-verification bridge. A JSONL event stream + atomic

status snapshot pattern is mandated by §11.4.116 (the exact HelixQA emitter file not read this session — see GAPS).

GAPS (§6): the concrete JSONL event-stream/status-snapshot file for the conductor ↔ framework sync channel (§11.4.116) was not located this session (checked cmd/ names only, not pkg//internal/).

7. **claude_toolkit** (SIBLING project — /home/milos/Factory/projects/tools_and_research/claude_toolkit)

Outside the monorepo. It is the **provider-alias generator for CLI agents** (Claude Code + OpenCode).

- **Provider-alias mechanism** = `scripts/claude-providers.sh (sync | list | show | remove | add):` > “create/refresh/list/remove Claude Code aliases for non-Anthropic LLM providers, fully dynamically. > Pipeline (sync): read the API-key VARIABLE NAMES from the keys file, fetch + cache the **models.dev** > catalog, resolve each LLM key into a concrete provider record (provider id, alias, base URL, transport, > strong/fast model) via `providers_resolve.py`, optionally **verify with LLMsVerifier**, then generate > for each provider: a non-secret env file, a shell alias (`cma_run_provider <id>`), a config dir > (`~/.claude-prov-<id>`) linking all shared items, and the always-on plugin set. Idempotent.” > “Nothing about providers/models is hardcoded — everything derives from `models.dev` + editable > `providers/key-aliases.json` and `overrides.json`.” (`scripts/claude-providers.sh:1-24`)
- **Alias generation:** `scripts/providers_generate.py` reads `verified_models.json` (from `model_verify.py`), sorts models by score, pairs 2/alias (strong + fast), names provider, provider2, ... (`providers_generate.py:1-70`).
- **Binary-on-PATH → alias:** the mechanism keys off **API-key env-var names present in the environment/keys file**, not off detecting a binary on PATH. Provider detection = “which key variables are set” → resolve via `models.dev` + `providers_resolve.py`. (No which <binary>-style PATH sniff found in the read excerpt; a local-server provider like `llama.cpp` would need a key-alias/override entry.)
- **Vendored LLMsVerifier:** `claude_toolkit/submodules/LLMsVerifier` (its `.gitmodules:url` = `git@github.com:vasic-digital/LLMsVerifier.git`, branch `main`) — SAME upstream as the monorepo’s `submodules/llms_verifier`. Also vendors `submodules/containers` and `submodules/challenges`.

- Other scripts: `providers_resolve.py`, `providers-verify.sh`, `model_verify.py`, `claude-verify-providers.sh`, `providers-semantic.sh`, `opencode_sync.py`, `toon_encode.py`, plus scripts/`providers/{key-aliases.json, overrides.json, evidence/models-dev-mapping.json, rubric/, fixture/}`.

GAPS (§7): no local-server (`llama.cpp`/HelixLLM) provider entry found in `providers/key-aliases.json` excerpt — to expose a local HelixLLM/`llama.cpp` endpoint as a Claude-Code alias, a new key-alias/override + base-URL record would be needed. PATH-based binary auto-detection is NOT the current mechanism (env-key driven).

8. CodeGraph + OpenDesign wiring

- **CodeGraph:** wired in root `.mcp.json` as MCP server codegraph → command: `/Users/milosvasic/.local/bin/codegraph serve --mcp --path /Volumes/T7/Projects/helix_code`. **NOTE the hardcoded macOS paths** (`/Users/milosvasic`, `/Volumes/T7`) — this `.mcp.json` targets a macOS host, not the current Linux host (`/home/milos/...`). Root `.codegraph/` dir exists. Own-org submodules should be indexed per §11.4.79.
- **OpenDesign:** wired in root `.mcp.json` as MCP server open-design → `npx -y open-design-mcp`, `OD_DAEMON_URL: http://localhost:7456`, **"disabled": true**. Docs: `docs/OPENDESIGN.md`, audit `docs/research/opendesign_audit_20260622/findings.md`. §11.4.162 mandate exists but the MCP is currently disabled.
- **media-validator** MCP is also wired (enabled) → `constitution/skills/media-validator/media-validator.sh` (§11.4.164).

GAPS (§8): `.mcp.json` contains **macOS-absolute paths** that will not resolve on this Linux host (CodeGraph binary path + `--path`); OpenDesign MCP is disabled. Both need host-correct wiring before use.

9. Build / run / PATH

- **HelixLLM** (`submodules/helix_llm/Makefile`): `make build` (→ binary), `make dev` (certs + run), `make container`, `test-unit`, `test-integration`, `test-race`, `test-stress`, `test-chaos`, `test-security`, `test-challenges`, `test-automation`, `deploy`, `ingest`, `probe`, `scan-*`, `lint`, `fmt`. Docker facade: `docker-compose.enterprise.yml`.
- **HelixAgent** (`submodules/helix_agent/`): many `docker-compose.*.yml` incl. `docker-compose.helixllm.yml` + `docker-compose.helixllm-`

infra.yml; Makefile present; per-CLI-agent challenge scripts under challenges/scripts/.

- **Meta-repo root:** setup.sh (submodule init + deps + build), Makefile (governance gates), helix facade script, compose.helixcode-infra.yml, docker-compose.helix.yml. Root go.mod is the thin dev.helix.code module.
- **Inner Go app** helix_code/ (tracked subdir, module dev.helix.code): cmd/{server,cli, helix-config,...}; CLI reads HELIX_LLM_PROVIDER env (helix_code/cmd/cli/main.go:192,599,1257).
- **How binaries reach PATH:** via each submodule's make build → local bin/; there is no global PATH-install recipe surfaced. Container path uses the root ./helix facade + compose files (§11.4.161 mandates rootless Podman via the containers submodule).

GAPS (§9): - No single top-level target builds “llama.cpp on the host GPU → helix_llm server → helix_agent → CLI aliases” end-to-end; today it is three separate submodule builds + docker-compose files + the sibling claude_toolkit alias generator. - llama.cpp host build (from vendored dependencies/LLama_CPP) is not wired into setup.sh/any Makefile target found this session.

GAPS SUMMARY — top integration gaps for the goal

(run local models via llama.cpp on RTX 5090 → expose through HelixAgent to HelixCode/CLI agents → add vision/translation/embeddings/RAG/MCP/ACP)

1. **No GPU/RTX-5090 build+launch wiring for llama.cpp.** Vendored source (dependencies/LLama_CPP) is present but no host-GPU build recipe or -ngl/tensor-split launch config found; HelixLLM's LlamaServerConfig GPU fields unverified (§4, §9).
2. **No vision or translation serving surface** in either HelixLLM or HelixAgent (only embeddings + RAG exist). LLMsVerifier Capabilities has Multimodal/ImageGeneration/Voice booleans but no serving path consumes them (§1, §2, §3).
3. **HelixLLM is not a pinned own-org submodule at the meta-repo root** — no .gitmodules entry for helix_llm; two on-disk checkouts (submodules/helix_llm + dependencies/HelixDevelopment/helix_llm) risk drift (§1).
4. **LLMsVerifier capability flags are incomplete for CONST-040** — no model-level MCP/LSP/ACP/RAG/Skills/Plugins booleans on VerificationResult; local-provider (llama.cpp/ollama) verification not URL-matched (§3).
5. **Host-wiring rot in .mcp.json** — CodeGraph points at macOS-absolute paths (/Users/..., /Volumes/T7/...), OpenDesign MCP disabled; neither works on the current Linux host without re-wiring (§8). Plus: claude_toolkit exposes providers by **env-key**, not by detecting a local binary/endpoint on PATH — a

local HelixLLM/llama.cpp alias needs a new key-alias/override + base-URL record (§7).